

Модуль 8. Тестирование асинхронных приложений

Введение в модуль

Код, который нельзя протестировать, нельзя и доверять. В асинхронном мире тестирование обрывает дополнительную сложностью: корутины требуют event loop, моки должны различать синхронные и асинхронные вызовы, а race condition воспроизводятся только под нагрузкой. В этом модуле мы построим полную пирамиду тестирования для FastAPI-приложения — от изолированных unit-тестов отдельных корутин до нагрузочных испытаний, показывающих, где именно ломается event loop.

Мы разберём:

- почему пирамида тестирования для микросервисов превращается в бриллиант;
- как `pytest-asyncio` управляет event loop и почему режим `auto` опасен;
- как мокать асинхронные зависимости, не ломая семантику `await` ;
- как поднимать тестовую базу данных, откатывать транзакции и генерировать фейковые данные;
- что такое p99 latency и почему среднее значение — ложный друг;
- как профилировать асинхронный Python, чтобы найти точку, где корутина блокирует цикл.

8.1. Пирамида тестирования

8.1.1. Три уровня: Unit, Integration, E2E

Unit-тест (модульный тест) проверяет изолированный кусок кода — функцию, метод, класс — вне контекста всего приложения. Зависимости подменяются моками.

```
In [1]: # Что тестируем: чистая функция валидации
def validate_email(email: str) -> bool:
    return "@" in email and "." in email.split("@")[1]

# Unit-тест
def test_validate_email():
```

```
assert validate_email("alice@example.com") is True
assert validate_email("invalid") is False
```

Интеграционный тест проверяет взаимодействие нескольких компонентов: endpoint + база данных, сервис + внешний API, репозиторий + ORM.

```
In [2]: # Что тестируем: endpoint обращается к реальной (тестовой) БД
def test_create_user(client, db):
    response = client.post("/users/", json={"name": "Alice", "email": "a@b.com"})
    assert response.status_code == 201
    assert db.query(User).count() == 1
```

E2E (End-to-End) тест проверяет всю систему сквозь: от HTTP-запроса через балансировщик до записи в БД и обратно. E2E тесты запускаются в окружении, максимально приближенном к продакшену.

8.1.2. Границы ответственности

Уровень	Скорость	Стоимость поддержки	Что ловит
Unit	Миллисекунды	Низкая	Ошибки логики, граничные условия
Integration	Секунды	Средняя	Ошибки взаимодействия компонентов, SQL, сериализация
E2E	Минуты	Высокая	Ошибки конфигурации, инфраструктуры, сети

Принцип: чем выше уровень, тем медленнее и хрупче тест. Поэтому большинство тестов должны быть unit.

8.1.3. Пирамида vs бриллиант для микросервисов

Классическая **пирамида** (Майк Кон, 2009):

- Основание: много unit-тестов.
- Середина: меньше интеграционных.
- Вершина: немного E2E.

Но в микросервисной архитектуре unit-тест отдельного сервиса проверяет лишь малую часть логики. Основная сложность — в **взаимодействии** сервисов (контракты API, версионирование, сетевые сбои). Поэтому для микросервисов пирамида часто превращается в **бриллиант** (test diamond):

- Много unit-тестов для внутренней логики.
- **Ещё больше** интеграционных тестов (сервис + его БД, кэш, брокер).
- Много контрактных тестов (consumer-driven contracts, Pact).
- Умеренное число E2E.

Для ML-сервиса на FastAPI это означает:

- Unit: валидация Pydantic, бизнес-логика предобработки данных.
- Integration: endpoint + SQLAlchemy + тестовая PostgreSQL, endpoint + Redis.
- E2E: полный сценарий «загрузка файла -> inference -> запись в БД -> получение результата».

8.2. Unit-тесты

8.2.1. pytest-asyncio: управление event loop

Стандартный `pytest` не умеет `await`. Плагин `pytest-asyncio` добавляет эту возможность.

Установка и базовый синтаксис

```
In [3]: pip install pytest-asyncio
```

```
In [4]: import pytest

@pytest.mark.asyncio
async def test_async_add():
    result = await async_add(2, 3)
    assert result == 5
```

`@pytest.mark.asyncio` сообщает pytest: «эта корутина должна быть запущена в event loop».

Режимы работы: `strict` vs `auto`

В `pytest-asyncio` есть два режима:

strict (по умолчанию в v0.21+):

- Только функции с `@pytest.mark.asyncio` запускаются в event loop.
- Явный контроль. Рекомендуется для новых проектов.

auto:

- Все `async def` тесты автоматически считаются асинхронными.
- Опасен: если случайно импортировать `async def` функцию из другого модуля в тестовый файл, pytest попытается запустить её как тест.

Настройка в `pyproject.toml` :

```
In [5]: [tool.pytest.ini_options]
        asyncio_mode = "strict"
```

Fixture `event_loop`

`pytest-asyncio` предоставляет фикстуру `event_loop` . Её можно переопределить, чтобы контролировать политику создания цикла:

```
In [6]: import asyncio
        import pytest

        @pytest.fixture
        def event_loop():
            # Создаём новый цикл для каждого теста
            loop = asyncio.get_event_loop_policy().new_event_loop()
            yield loop
            loop.close()
```

Важно: в `pytest-asyncio` v0.21+ рекомендуется использовать `asyncio.run()` -подобную семантику: каждый тест получает **свежий** event loop. Это изолирует тесты друг от друга. Если один тест регистрирует фоновую задачу и не отменяет её, она не «просочится» в следующий тест.

Scope фикстур и event loop

По умолчанию `event_loop` имеет `scope="function"` . Если нужно разделить цикл между несколькими тестами (например, для пула соединений с БД):

```
In [7]: @pytest.fixture(scope="module")
def event_loop():
    loop = asyncio.get_event_loop_policy().new_event_loop()
    yield loop
    loop.close()
```

Осторожно: общий event loop между тестами может привести к состоянию гонки.

Используйте `scope="module"` только для тяжёлых ресурсов, которые дорого создавать на каждый тест (например, тестовый контейнер PostgreSQL).

8.2.2. Моки и стабы: `unittest.mock`, `pytest-mock`, `respx`

`unittest.mock` : ОСНОВЫ

Модуль `unittest.mock` входит в стандартную библиотеку Python.

```
In [8]: from unittest.mock import Mock, AsyncMock, patch

# Синхронный мок
mock = Mock()
mock.return_value = 42
assert mock() == 42

# Асинхронный мок
async_mock = AsyncMock()
async_mock.return_value = {"status": "ok"}
assert await async_mock() == {"status": "ok"}
```

`Mock` vs `AsyncMock` :

- `Mock` — синхронный. Если присвоить его асинхронной функции и вызвать с `await`, получите `TypeError: object Mock can't be used in 'await' expression`.
- `AsyncMock` — возвращает awaitable. Корректно работает с `await`.

`pytest-mock` : интеграция с `pytest`

```
In [9]: pip install pytest-mock
```

Предоставляет фикстуру `mockers` :

```
In [10]: def test_fetch_data(mock):
# Подменяем функцию в модуле myapp.services
mock_get = mock.patch("myapp.services.http_get", new_callable=AsyncMock)
mock_get.return_value = {"data": "test"}

result = asyncio.run(fetch_data("http://example.com"))
assert result == {"data": "test"}
mock_get.assert_awaited_once_with("http://example.com")
```

Критически важно: патчить нужно там, где используется, а не где определено.

```
In [11]: # myapp/services.py
from aiohttp import ClientSession

async def fetch():
    async with ClientSession() as session:
        return await session.get("http://api.com")

# test.py
# ПЛОХО: патчим aiohttp в его собственном модуле
mock.patch("aiohttp.ClientSession.get")

# ХОРОШО: патчим в модуле, который использует session.get
mock.patch("myapp.services.ClientSession.get")
```

Почему? Потому что Python импортирует объект по ссылке. Когда `myapp.services` делает `from aiohttp import ClientSession`, в `myapp.services` создаётся **своя** ссылка на `ClientSession`. Патч `aiohttp.ClientSession` не затронет эту ссылку.

respx : мок HTTPX-запросов

```
In [12]: pip install respx
```

```
In [13]: import respx
from httpx import Response

@respx.mock
async def test_api_client():
    route = respx.get("https://api.example.com/data").mock(return_value=Response(
        200, json={"key": "value"}))
```

```
async with httpx.AsyncClient() as client:
    response = await client.get("https://api.example.com/data")

assert response.json() == {"key": "value"}
assert route.called
```

`respx` перехватывает HTTP-запросы на уровне `httpx`, не отправляя их в сеть. Это быстрее и надёжнее, чем мокать методы вручную.

8.2.3. Тестирование корутин, таймаутов, исключений

Корутины с `asyncio.gather`

```
In [14]: @pytest.mark.asyncio
async def test_gather_results():
    async def task(n):
        await asyncio.sleep(0.001)
        return n * 2

    results = await asyncio.gather(task(1), task(2), task(3))
    assert results == [2, 4, 6]
```

Таймауты

```
In [15]: import asyncio

@pytest.mark.asyncio
async def test_timeout_raises():
    async def slow():
        await asyncio.sleep(10)

    with pytest.raises(asyncio.TimeoutError):
        await asyncio.wait_for(slow(), timeout=0.1)
```

Исключения в корутинах

```
In [16]: @pytest.mark.asyncio
async def test_exception_in_task():
    async def failing():
```

```

        raise ValueError("boom")

    task = asyncio.create_task(failing())

    with pytest.raises(ValueError, match="boom"):
        await task

```

Тестирование `async for`

```

In [17]: @pytest.mark.asyncio
        async def test_async_generator():
            async def gen():
                for i in range(3):
                    yield i

            result = [x async for x in gen()]
            assert result == [0, 1, 2]

```

8.2.4. Паттерн Arrange-Act-Assert для асинхронного кода

Структура каждого теста:

```

In [18]: @pytest.mark.asyncio
        async def test_user_service_creates_user():
            # ARRANGE: подготовка данных и моков
            mock_repo = AsyncMock()
            mock_repo.create.return_value = User(id=1, name="Alice")
            service = UserService(mock_repo)

            # ACT: вызов тестируемого кода
            result = await service.create_user(name="Alice")

            # ASSERT: проверка результата и побочных эффектов
            assert result.name == "Alice"
            mock_repo.create.assert_awaited_once_with(name="Alice")

```

Почему `assert_awaited_once_with`, а не `assert_called_once_with` ?

- `assert_called` — проверяет, что метод вызван (для синхронных).
- `assert_awaited` — проверяет, что метод был **дождаться** (`await ed`). Для `AsyncMock` это семантически корректнее.

8.2.5. Математическая подоплека: покрытие кода и комбинаторика

Покрытие кода (Code Coverage) — метрика, показывающая, какая доля исходного кода была выполнена тестами. Измеряется инструментами (`pytest-cov` , `coverage.py`).

Виды покрытия:

- **Statement coverage:** процент выполненных строк. Слабый: `if a or b` покрыт, если `a=True` , даже если `b` никогда не проверялось.
- **Branch coverage:** процент выполненных ветвлений (true/false для каждого условия). Сильнее.
- **Path coverage:** процент выполненных путей через граф потока управления. Идеален, но экспоненциально дорог.

Для функции с n независимыми условиями число путей:

$$\text{paths} = 2^n$$

Полное path coverage невозможно для сложных функций. Поэтому используют **MC/DC (Modified Condition/Decision Coverage)** — каждое условие должно быть проверено независимо. Для `if a or b` нужно три теста: `(T, *)` , `(F, T)` , `(F, F)` .

Для асинхронного кода добавляются дополнительные измерения:

- Покрытие точек `await` (все ли `await` были достигнуты?).
- Покрытие обработчиков исключений (все ли `except` срабатывали?).
- Покрытие callback'ов и задач (все ли `create_task` завершались успешно и с ошибкой?).

8.3. Интеграционные тесты

8.3.1. `TestClient` от Starlette

`TestClient` — синхронный клиент для ASGI-приложения. Он запускает ваше приложение в отдельном потоке и отправляет HTTP-запросы.

```
In [19]: from fastapi.testclient import TestClient
         from main import app

         client = TestClient(app)

         def test_read_main():
             response = client.get("/")
```

```
assert response.status_code == 200
assert response.json() == {"message": "Hello World"}
```

Важно: `TestClient` синхронен. Внутри он использует `requests` и `httpx` для запуска ASGI-приложения в фоновом event loop. Это удобно, но:

- Не тестирует реальную асинхронность (все запросы в `TestClient` идут последовательно в одном тесте).
- Не подходит для тестирования конкурентных сценариев (два одновременных запроса к одному ресурсу).

8.3.2. `httpx.AsyncClient` для асинхронных интеграционных тестов

Для полноценного тестирования асинхронности:

```
In [20]: import pytest
        from httpx import AsyncClient

        @pytest.fixture
        async def async_client():
            async with AsyncClient(app=app, base_url="http://test") as client:
                yield client

        @pytest.mark.asyncio
        async def test_create_user(async_client):
            response = await async_client.post("/users/", json={"name": "Alice"})
            assert response.status_code == 201
```

Преимущества:

- Реальный `await` на уровне теста.
- Возможность запускать конкурентные запросы через `asyncio.gather`.
- Тестирование streaming-ответов.

```
In [21]: @pytest.mark.asyncio
        async def test_concurrent_requests(async_client):
            responses = await asyncio.gather(
                *[async_client.get("/items/") for _ in range(10)]
            )
            assert all(r.status_code == 200 for r in responses)
```

8.3.3. Тестовая БД: транзакции с откатом

Интеграционные тесты должны работать с **реальной** СУБД (чтобы ловить особенности PostgreSQL, например, `RETURNING` или `FOR UPDATE`), но не должны засорять данные.

Стратегия 1: откат транзакции после каждого теста

```
In [22]: import pytest
from sqlalchemy.ext.asyncio import AsyncSession, async_sessionmaker

@pytest.fixture
async def db_session():
    # Создаём новую сессию
    async with async_sessionmaker(engine, expire_on_commit=False)() as session:
        # Начинаем вложенную транзакцию
        async with session.begin_nested():
            yield session
        # После yield откатываем вложенную транзакцию
        await session.rollback()
```

Как это работает:

- `BEGIN` — внешняя транзакция.
- `SAVEPOINT` — точка сохранения (вложенная транзакция, `begin_nested()` в SQLAlchemy).
- Тест работает с данными внутри savepoint.
- После теста `ROLLBACK TO SAVEPOINT` — все изменения исчезают.
- Внешняя транзакция тоже откатывается.

Нюанс: `asyncpg` и PostgreSQL поддерживают вложенные транзакции через savepoints. SQLite — нет (или с ограничениями). Поэтому для интеграционных тестов рекомендуется использовать **тот же движок**, что и в продакшене (PostgreSQL).

Стратегия 2: тестовый контейнер PostgreSQL

```
In [23]: import pytest_asyncio
from testcontainers.postgres import PostgresContainer

@pytest_asyncio.fixture(scope="session")
async def postgres_container():
    with PostgresContainer("postgres:16-alpine") as postgres:
        yield postgres.get_connection_url(driver="asyncpg")
```

```

@pytest_asyncio.fixture
async def db_engine(postgres_container):
    engine = create_async_engine(postgres_container)
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield engine
    await engine.dispose()

```

`testcontainers` запускает Docker-контейнер с PostgreSQL на время тестовой сессии. Это медленнее, но максимально приближено к реальности.

8.3.4. Фабрики данных: `factory-boy`, `faker`

Ручное создание тестовых данных утомительно и хрупко:

```

In [24]: # Плохо: при изменении схемы придётся менять все тесты
user = {"id": 1, "name": "Alice", "email": "a@b.com", "created_at": "2024-01-01"}

```

Factory Boy + **Faker** автоматизируют генерацию:

```

In [25]: pip install factory-boy faker

```

```

In [26]: import factory
from factory.faker import Faker as FactoryFaker
from myapp.models import User

class UserFactory(factory.Factory):
    class Meta:
        model = User

    id = factory.Sequence(lambda n: n)
    name = FactoryFaker("name")
    email = FactoryFaker("email")
    is_active = True

# Использование
user = UserFactory.build() # создаёт объект, не сохраняя в БД
user = UserFactory.create() # сохраняет в БД (для синхронных фабрик)

```

Для асинхронных моделей SQLAlchemy:

```
In [27]: class AsyncUserFactory(factory.Factory):
        class Meta:
            model = User

        @classmethod
        async def create(cls, **kwargs):
            obj = cls.build(**kwargs)
            db.add(obj)
            await db.flush()
            return obj

# В тесте
user = await AsyncUserFactory.create(name="Specific Name")
```

Faker генерирует локализованные данные:

```
In [28]: from faker import Faker

fake = Faker("ru_RU")
fake.name()    # 'Иванов Иван Иванович'
fake.address() # 'г. Москва, ул. Ленина, д. 1'
fake.email()   # 'ivanov@example.com'
```

8.3.5. Тестирование DI и переопределение зависимостей

Вспомним Модуль 5: `app.dependency_overrides` используется для подмены зависимостей в тестах.

```
In [29]: @pytest.fixture
        def client_with_fake_db():
            app.dependency_overrides[get_db] = lambda: FakeDB()
            with TestClient(app) as client:
                yield client
            app.dependency_overrides.clear()

        def test_endpoint(client_with_fake_db):
            response = client_with_fake_db.get("/items/")
            assert response.status_code == 200
```

Важно: `dependency_overrides` — глобальное состояние приложения. Если не очищать после теста, переопределения «просочатся» в другие тесты. Используйте `yield`-fixture с очисткой.

8.3.6. Математическая подоплека: интеграционное тестирование как проверка композиции

В теории категорий композиция морфизмов $f : A \rightarrow B$ и $g : B \rightarrow C$ даёт $g \circ f : A \rightarrow C$.

Интеграционный тест проверяет, что композиция компонентов сохраняет **инварианты** каждого компонента. Формально, если f и g — частичные функции с предусловиями P_f, P_g и постусловиями Q_f, Q_g , то композиция $g \circ f$ имеет:

- Предусловие: P_f (и P_g должно выполняться для результата f).
- Постусловие: Q_g .

Интеграционный тест ловит ситуации, когда $Q_f \not\Rightarrow P_g$ — выход одного компонента не удовлетворяет входным требованиям другого.

Пример: Pydantic-модель ожидает `email: str`, а репозиторий возвращает `None` при отсутствии записи. Если endpoint не обрабатывает `None` — интеграционный тест поймает `AttributeError`, хотя unit-тесты репозитория и Pydantic-модели проходят по отдельности.

8.4. Нагрузочное тестирование

8.4.1. Зачем нагружать ML-сервис?

Unit и интеграционные тесты проверяют корректность. Нагрузочные тесты проверяют **адекватность**:

- Сколько RPS выдержит endpoint inference до деградации?
- Как растёт latency при увеличении concurrency?
- Где узкое место: CPU (inference), память (модель), I/O (БД), event loop (блокировки)?

8.4.2. Инструменты

`wrk` — простой и быстрый

```
In [30]: wrk -t12 -c400 -d30s http://localhost:8000/predict
```

- `-t12` : 12 потоков.

- `-c400` : 400 соединений.
- `-d30s` : 30 секунд.

Выводит средний RPS, latency, ошибки. Не поддерживает POST с телом из коробки (нужен Lua-скрипт).

`vegeta` — HTTP-нагрузка как код

```
In [31]: echo "POST http://localhost:8000/predict" | vegeta attack -duration=30s -rate=1000 | vegeta report
```

Поддерживает постоянную нагрузку (constant rate), что важно для проверки стабильности. Выдаёт распределение latency.

`locust` — Python, программируемый

```
In [32]: from locust import HttpUser, task, between

class MLUser(HttpUser):
    wait_time = between(1, 3) # пауза между запросами 1-3 секунды

    @task(3)
    def predict(self):
        self.client.post("/predict", json={"features": [1.0, 2.0, 3.0]})

    @task(1)
    def health(self):
        self.client.get("/health")
```

Запуск:

```
In [33]: locust -f locustfile.py --host=http://localhost:8000
```

Web-интерфейс Locust показывает RPS, latency, распределение в реальном времени. Можно масштабировать на кластер (master + workers).

`k6` — современный, JavaScript, CI-friendly

```
In [34]: import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 }, // ramp up
    { duration: '5m', target: 100 }, // steady
    { duration: '2m', target: 200 }, // ramp up
    { duration: '5m', target: 200 }, // steady
    { duration: '2m', target: 0 }, // ramp down
  ],
};

export default function () {
  const res = http.post('http://localhost:8000/predict', JSON.stringify({
    features: [1.0, 2.0, 3.0]
  }), { headers: { 'Content-Type': 'application/json' } });

  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Stages моделируют реалистичную нагрузку: плавный рост, плато, пик, спад. Это важнее, чем мгновенный всплеск — позволяет увидеть, как система адаптируется (например, как ведёт себя connection pool).

8.4.3. Метрики: RPS, latency, ошибки, ресурсы

RPS (Requests Per Second)

$$\text{RPS} = \frac{\text{число успешных запросов}}{\text{время теста (сек)}}$$

Не путать с **throughput** — пропускной способностью в байтах/сек.

Latency: p50, p95, p99

Latency — время от отправки запроса до получения ответа. **Никогда не используйте среднее (mean)** для оценки latency.

Почему? Распределение latency имеет **тяжёлый правый хвост**: большинство запросов быстрые, но некоторые (GC-пауза, блокировка в БД, JIT-компиляция) — очень медленные. Среднее скрывает проблемы.

Персентиль	Смысл
p50 (median)	50% запросов быстрее этого значения. Типичный опыт пользователя.
p95	95% запросов быстрее. «Худший нормальный случай».
p99	99% запросов быстрее. Выбросы, но не катастрофа.
p99.9	0.1% запросов медленнее. Критично для финансовых систем.

Для ML-сервиса: если p95 inference — 100 мс, а p99 — 5 секунд — значит 1% запросов попадает в проблему (возможно, cold start модели или GC).

Ошибки

- **HTTP 5xx:** ошибки сервера (перегрузка, баги).
- **HTTP 4xx:** ошибки клиента (невалидный JSON, rate limit).
- **Таймауты:** запрос не получил ответ в заданное время.
- **Connection errors:** сервер отказал в соединении (пул исчерпан).

SLO (Service Level Objective): целевые метрики. Например:

- $RPS \geq 1000$.
- $p95 \text{ latency} \leq 200 \text{ мс}$.
- $\text{Error rate} \leq 0.1\%$.

8.4.4. Профилирование

py-spy — сэмплирующий профилировщик

```
In [35]: pip install py-spy
py-spy top --pid $(pgrep -f uvicorn)
py-spy record -o profile.svg --pid $(pgrep -f uvicorn)
```

Работает извне процесса, не требует модификации кода, не влияет на производительность существенно. Показывает, какие функции занимают больше всего CPU времени.

memray — профилировщик памяти

```
In [36]: pip install memray
         memray run -o output.bin -m uvicorn main:app
         memray flamegraph output.bin
```

Показывает, где утекает память (например, кэш без eviction, или накопление объектов в очереди).

asyncio debug mode

```
In [37]: import asyncio
         asyncio.run(main(), debug=True)
```

Или через переменную окружения:

```
In [38]: PYTHONASYNCIODEBUG=1 uvicorn main:app
```

В debug mode:

- Корутины, которые выполняются дольше 100 мс без уступки управления, логируются как «blocking».
- Незакрытые задачи (tasks not awaited) вызывают предупреждения.
- Неотменённые `asyncio.gather` с исключениями показывают traceback.

Критично для ML: если inference блокирует event loop на 2 секунды, debug mode покажет:

```
In [39]: Executing <Task finished name='Task-1' coro=<predict() done, defined at main.py:4
         2>
         created at asyncio/tasks.py:384> took 2.005 seconds
```

8.4.5. Математическая подоплека: распределения latency и теория очередей

Распределение latency

Latency веб-запросов редко подчиняется нормальному распределению. Чаще это **логнормальное** или **экспоненциальное** распределение.

Экспоненциальное распределение (характерно для «без памяти» процессов, например, времени между запросами в пуассоновском потоке):

$$f(x; \lambda) = \lambda e^{-\lambda x}, \quad x \geq 0$$

Среднее: $\frac{1}{\lambda}$. Медиана: $\frac{\ln 2}{\lambda}$. Дисперсия: $\frac{1}{\lambda^2}$.

Парето-распределение (для тяжёлых хвостов):

$$f(x) = \frac{\alpha x_m^\alpha}{x^{\alpha+1}}, \quad x \geq x_m$$

Параметр α определяет «тяжесть» хвоста. При $\alpha \leq 2$ дисперсия бесконечна — среднее не имеет смысла. Именно поэтому для latency используют перцентили, а не среднее.

Закон Литтла (Little's Law)

Фундаментальная теорема теории очередей:

$$L = \lambda W$$

где:

- L — среднее число запросов в системе (очередь + обслуживание).
- λ — интенсивность поступления (RPS).
- W — среднее время пребывания запроса в системе (latency).

Следствие: если ваш сервер обрабатывает $\lambda = 1000$ RPS со средней задержкой $W = 0.1$ с, то в любой момент в системе находится $L = 100$ запросов. Если connection pool рассчитан на 50 соединений — система перегружена.

Модель M/M/1 для оценки capacity

Для одного CPU-ядра с одним event loop (что соответствует одному worker'у Uvicorn) и пуассоновским потоком запросов:

$$\rho = \lambda \cdot S$$

где S — среднее время обслуживания одного запроса.

Среднее число запросов в очереди:

$$L_q = \frac{\rho^2}{1 - \rho}, \quad \rho < 1$$

Среднее время ожидания в очереди:

$$W_q = \frac{\rho \cdot S}{1 - \rho}$$

Ключевой вывод: при $\rho \rightarrow 1$ (загрузка стремится к 100%) время ожидания стремится к бесконечности. Поэтому нагрузочное тестирование ищет **точку насыщения** — максимальную λ , при которой W_q ещё приемлемо. Обычно это $\rho \approx 0.7 - 0.8$.

Для ML-сервиса: если inference занимает $S = 50$ мс, то один worker теоретически способен обработать $\lambda_{max} = \frac{1}{S} = 20$ RPS без очереди. На практике, из-за накладных расходов I/O, безопасный лимит — 14–16 RPS на worker. Для 1000 RPS нужно $\frac{1000}{15} \approx 67$ worker'ов (или процессов в Gunicorn).

Итог модуля 8

Концепция	Суть	Практическое применение
Пирамида тестирования	Больше unit, меньше E2E	Для микросервисов — бриллиант (много интеграционных)
pytest-asyncio	Запуск корутин в тестах	Режим <code>strict</code> , фикстура <code>event_loop</code>
AsyncMock	Мок для асинхронных функций	<code>assert_awaited_once_with</code> вместо <code>assert_called</code>
respx	Перехват HTTPX-запросов	Тестирование клиентов внешних API
TestClient	Синхронный клиент для ASGI	Быстрые интеграционные тесты
AsyncClient	Асинхронный клиент для ASGI	Конкурентные тесты, streaming
Rollback-транзакции	Savepoint + rollback после теста	Чистая тестовая БД без пересоздания
Factory Boy + Faker	Генерация тестовых данных	Поддерживаемые тесты при изменении схемы
p99 latency	99% запросов быстрее этого значения	SLO для ML API
Locust/k6	Нагрузочное тестирование	Проверка поведения под нагрузкой

Концепция	Суть	Практическое применение
py-spy / memray	Профилировщики CPU и памяти	Поиск блокировок и утечек
asyncio debug	Предупреждения о блокировках	Обнаружение синхронных вызовов в event loop
Закон Литтла	$L = \lambda W$	Оценка необходимого числа worker'ов

В **Модуле 9** мы перейдём к интеграции ML в асинхронный бэкенд: архитектурные паттерны inference, фоновые задачи, streaming через WebSocket и SSE, feature stores.